Trabajo Práctico Nº 1

SimpsonCraft

Fecha presentación	11/09/2025
Fecha entrega	09/10/2025

Segundo cuatrimestre 2025

1. Introducción

Homero está muy enojado desde que se enteró que Moe le robó la idea del trago que él inventó y lo está promocionando como "Llamarada Moe" y no como "Llamarada Homero". Esta noche, decidido a vengarse, Homero va a interrumpir en la fiesta que va a dar Moe. Una vez ahí, va a revelarles a todos el ingrediente secreto de la bebida, arruinando no sólo la celebración, sino también el millonario contrato que Moe estaba a punto de firmar con una importante empresa.

2. Objetivo

El presente trabajo práctico tiene como objetivo que el alumno:

1. Diseñe y desarrolle las funcionalidades de una biblioteca con un contrato preestablecido.
2. Se familiarice con y utilice correctamente los tipos de datos estructurados.
3. Desarrolle una interfaz gráfica amigable y entendible para el usuario.

Por supuesto, se requiere que el trabajo cumpla con las **buenas prácticas de programación** profesadas por la cátedra. Se considerarán críticos la modularización, reutilización y claridad del código.

3. Enunciado

El juego consiste en ayudar a Homero a encontrar su receta original para revelar el ingrediente secreto y luego entrar a la fiesta de Moe, antes de quedarse sin movimientos.

El juego se dará por ganado cuando Homero se encuentre en la misma posición que Moe, siempre y cuando, tenga movimientos disponibles y haya recolectado la receta.

El juego se dará por perdido si Homero se queda sin movimientos antes de llegar a Moe.

Al comenzar el juego, se deberán inicializar todos los elementos en posiciones aleatorias del terreno.

IMPORTANTE Ningún elemento puede inicializarse por fuera de los límites del terreno ni pisar otros elementos ya inicializados, con excepción de las herramientas y obstáculos sobre los bloques.

3.1. Homero

Homero comenzará desde la posición (0;0) y podrá moverse por todo el terreno, siempre y cuando esté parado **sobre** un bloque. De no haber bloque, el personaje caerá por gravedad hasta encontrarse con uno. Homero tendrá un total de 100 movimientos para poder llegar a la fiesta.

3.2. Moe

Estará posicionado en la última fila, sobre una columna random que no esté ocupada con ningún otro elemento.

3.3. Receta

Estará posicionada en un lugar aleatorio del terreno, que no esté ocupado con ningún otro elemento.

3.4. Bloques

Habrá 4 tipos de bloques a lo largo del mapa que le dificultarán o facilitarán el camino a Homero. El personaje siempre tiene que estar parado sobre un bloque, de caso contrario caerá hasta encontrar un bloque o hasta llegar al final del terreno.

Algunos bloques se pueden romper. La manera de romperlos es "chocándose" con ellos, avanzando hacia el mismo en la dirección en que se encuentra respecto al personaje. Es decir si hay un bloque a la derecha de Homero, este se tiene que mover a la derecha para chocarlo. Una vez que el bloque se rompa, ese espacio queda vacío para que el personaje en su próximo movimiento pueda ir hacia allí.

Cada bloque tendrá una resistencia y características diferentes:

3.4.1. Telarañas

Se deberán inicializar en posiciones aleatorias 100 bloques de telarañas.

Al ser débiles, se pueden romper chocándose con ellas una vez. Es decir, Homero gasta un movimiento para romper una telaraña.

Una vez rotas, las arañas se enojan y se van a armar su casa en otro lado, por lo que la telaraña deberá cambiar de posición.

3.4.2. Tierras

Se deberán inicializar en posiciones aleatorias 80 bloques de tierra.

Los bloques de tierra pueden romperse, pero para eso Homero deberá gastar dos movimientos. Una vez rotos, deben eliminarse del vector de bloques.

3.4.3. Vigas

Las vigas no pueden romperse por lo que su resistencia será -1. Homero únicamente podrá caminar sobre ellas. Para la implementación, la cátedra les facilitará una función que deberán llamar para tener las coordenadas donde deberán inicializarse (ver **Anexo 7.2**)

3.4.4. Escaleras

Las escaleras se pueden crear con maderas (ver sección de **Herramientas**). Le permiten a Homero subir, de forma que cuando se crea una escalera Homero queda sobre ella, y la escalera queda en la posición en la que estaba Homero, por lo que es importante chequear que en la posición donde quedaría Homero, no haya otros bloques.

Para crear una escalera, Homero necesita tener al menos 3 maderas en su mochila y **NO** gastará movimientos.

Al igual que las tierras y telarañas, las escaleras pueden romperse y le costará a Homero un movimiento. Al romper una escalera, el personaje recuperará 2 maderas.

3.5. Herramientas

Todas las herramientas deberán inicializarse en una posición aleatoria que esté ocupada por algún bloque, excepto **vigas**. De esta forma, las herramientas quedarán escondidas bajo los bloques y solo se revelará al usuario cuál hay una vez que se rompa el bloque. Una vez recolectadas, las herramientas tienen que eliminarse del vector.

Las herramientas no deben poder visualizarse en el terreno ya que están escondidas bajo un bloque.

Para recoger una herramienta el personaje tiene que colocarse encima de la misma.

3.5.1. Maderas

Las maderas le sirven a Homero para crear escaleras y poder subir, por lo que deberá juntar la mayor cantidad posible de maderas.

Al iniciar el juego, Homero tendrá 15 maderas disponibles en su mochila. Además, se tendrán que inicializar 20 maderas dispersas por el terreno.

3.5.2. Sopletes

Al iniciar el juego, Homero tendrá 2 sopletes disponibles en su mochila. Además, habrá 10 sopletes dispersos debajo de los bloques. Una vez recogidos, se guardan en la mochila. Le permitirán a Homero romper todos aquellos bloques que **NO** sean vigas a una distancia manhattan 2 (ver **Anexo 7.5**) del personaje.

Si al usar un soplete Homero cae por gravedad, debe juntar todos los objetos que se interpongan en su caída.

Usar un soplete **NO** gastará movimientos del personaje.

3.6. Obstáculos

Todos los obstáculos deberán inicializarse en una posición aleatoria que esté ocupada por algún bloque, excepto **vigas**. De esta forma, los obstáculos quedarán escondidos bajo los bloques y solo se revelará al usuario cuál hay una vez que se rompa el bloque.

Los obstáculos no deben poder visualizarse en el terreno ya que están escondidos bajo un bloque.

3.6.1. Cables

Se tendrán que inicializar 25 cables dispersos por el terreno en posiciones aleatorias.

Los cables electrocutan a Homero, por lo que en caso de encontrarse con uno, el personaje perderá 5 movimientos.

3.6.2. Murciélagos

Se tendrán que inicializar 20 murciélagos dispersos por el terreno en posiciones aleatorias.

Los murciélagos muerden las manos, por lo que le hará perder a Homero 3 maderas.

3.7. Terreno

El terreno será representado con una matriz de 30x15, donde estarán dispersos todos los elementos mencionados anteriormente.

3.8. Modo de juego

El objetivo del juego es que Homero recolecte la receta y luego llegue a Moe antes de quedarse sin movimientos.

Al moverse, Homero no puede pasarse de los límites del terreno. Por ejemplo, si Homero está en la columna 0 y el usuario lo mueve para la izquierda, Homero debería quedarse ahí,

no se debería mover porque estaría saliéndose del terreno. Ese movimiento no se restaría, ya que no se movió. Lo mismo ocurre si el personaje se quiere mover a través de una viga.

Es importante que antes de realizar el movimiento, se valide lo que ingrese el usuario, volviéndole a preguntar hasta que ingrese un movimiento correcto.

Homero se podrá mover libremente en 2 direcciones:

- **Derecha**: D
- **Izquierda**: A

Además, podrá romper bloques que estén a su izquierda o derecha y también los que estén debajo de él:

- **Abajo**: S

Si Homero no está parado sobre ningún tipo de bloque, entonces caerá por gravedad hasta que quede sobre uno.

Si Homero desea moverse hacia un costado donde hay un bloque, primero deberá romper el mismo y recién en el siguiente movimiento podrá moverse al lugar que quedó vacío. En caso de que al romper un bloque, Homero quede en el "aire", este caerá por gravedad en el mismo turno que lo rompió.

Para crear una escalera se deberá ingresar la siguiente instrucción:

- **Escalera**: E

Al ingresar esta instrucción, siempre y cuando se tengan maderas suficientes, Homero deberá subir una posición y debajo de él deberá visualizarse la escalera recién creada. Al crear una escalera, Homero pierde las maderas que gasta en ella, por lo que es importante que a lo largo del juego se recolecten la mayor cantidad de maderas.

Para activar un soplete se deberá ingresar la siguiente instrucción:

- **Soplete**: O

Únicamente se gastan movimientos al moverse o romper bloques, utilizar herramientas o caer por gravedad **no** gastan movimientos.

4. Especificaciones

4.1. Convenciones

- **Homero**: H
- **Moe**: M
- **Receta**: R

Herramientas:

- **Maderas**: W
- **Sopletes**: S

Obstaculos:

- **Cables**: C
- **Murcielagos**: B

Bloques:

- **Tierras**: T
- **Telarañas**: A

- **Vigas**: V
- **Escalera**: E

4.2. Orden de inicialización

1. Homero
2. Vigas
3. Moe
4. Tierras y telarañas
5. Objetos
6. Receta

4.3. Funciones y procedimientos

Para poder cumplir con lo pedido, se pedirá implementar las siguientes funciones y procedimientos.

El siguiente archivo **no** puede modificarse.

```
#ifndef __SIMPSONCRAFT_H__
#define __SIMPSONCRAFT_H__

#include <stdbool.h>

#define MAX_BLOQUES 400
#define MAX_OBJETOS 50
#define MAX_FILAS 30
#define MAX_COLUMNAS 15

typedef struct coordenada {
    int fil;
    int col;
} coordenada_t;

typedef struct personaje {
    char tipo;
    coordenada_t posicion;
    int cantidad_madera;
    int cantidad_sopletes;
    bool recolecto_receta;
} personaje_t;

typedef struct bloque {
    char tipo;
    coordenada_t posicion;
    int resistencia;
} bloque_t;

typedef struct objeto {
    char tipo;
    coordenada_t posicion;
} objeto_t;

typedef struct juego {
    personaje_t homero;
```

```
    coordenada_t posicion_moe;
    coordenada_t posicion_receta;
    int movimientos_restantes;
    bloque_t bloques[MAX_BLOQUES];
    int tope_bloques;
    objeto_t herramientas[MAX_OBJETOS];
    int tope_herramientas;
    objeto_t obstaculos[MAX_OBJETOS];
    int tope_obstaculos;
} juego_t;

/*
 * Pre condiciones: -
 * Post condiciones: Inicializará el juego, cargando toda la información
inicial de Homero, Moe, los bloques, las herramientas y los obstáculos.
 */
void inicializar_juego(juego_t *juego);


/*
 * Pre condiciones: El juego debe estar inicializado previamente con
`inicializar_juego` y la acción
 * debe ser válida.
 * Post condiciones: Realizará la acción recibida por parámetro actualizando el
juego.
 */
void realizar_jugada(juego_t *juego, char movimiento);

/*
 * Pre condiciones: El juego debe estar inicializado previamente con
`inicializar_juego `.
 * Post condiciones: Imprime el juego por pantalla.
 */
void mostrar_juego(juego_t juego);

/*
 * Pre condiciones: El juego deberá estar inicializado previamente con
`inicializar_juego `
 * Post condiciones: Devuelve:
 * --> 1 si es ganado
 * --> -1 si es perdido
 * --> 0 si se sigue jugando
 * El juego se dará por ganado cuando Homero recolecta su receta secreta y
llega a Moe.
 * Se dará por perdido si se le terminan los movimientos antes de recolectar la
receta y llegar a Moe.
 */
int estado_juego(juego_t juego);



#endif /* __SIMPSONCRAFT_H__ */
```

5. Resultado esperado

El tp es un juego. Esperamos que el trabajo cumpla la funcionalidad explicada anteriormente. Se deberá:

- Implementar todas las funciones especificadas en la biblioteca.
- Inicializar todos los campos del registro juego_t.
- Pedirle al usuario que ingrese una acción válida a realizar cada turno.
- Imprimir el terreno de forma clara con información que pueda serle útil al usuario (cuántos sopletes tiene, maderas, etc).
- Respetar las buenas prácticas de programación que profesamos en la cátedra.

6. Compilación y Entrega

El trabajo práctico debe ser realizado en un archivo llamado simpsoncraft.c, lo que sería la implementación de la biblioteca simpsoncraft.h. El trabajo debe ser compilado sin errores al correr el siguiente comando:

```
gcc *.c utiles.o -o juego -std=c99 -Wall -Wconversion -Werror -lm
```

Aclaración: Además del desarrollo de la biblioteca, se deberá implementar un main, el cual tiene que estar desarrollado en un archivo llamado juego.c. En este se manejará todo el flujo del programa, utilizando las funciones de la biblioteca realizada e incluyendo el pedido del movimiento al usuario y la validación del mismo. Queda a opción del alumno si desea crear otras bibliotecas, pero en caso de hacerlo, no deben ser más de dos.

Lo que nos permite ***.c** es agarrar todos los archivos que tengan la extensión .c en el directorio actual y los compile todos juntos. Esto permite que se puedan crear bibliotecas a criterio del alumno, aparte de las exigidas por la cátedra.

Por último debe ser entregado en la plataforma de corrección de trabajos prácticos **AlgoTrón** (patente pendiente), en la cual deberá tener la etiqueta **¡Exito!** significando que ha pasado las pruebas a las que la cátedra someterá al trabajo.

IMPORTANTE! Esto no implica necesariamente haber aprobado el trabajo ya que además será corregido por un colaborador que verificará que se cumplan las buenas prácticas de programación. Para que el trabajo sea corregido por un colaborador, el mismo debe pasar todas las pruebas mínimas..

Para la entrega en **AlgoTrón** (patente pendiente), recuerde que deberá subir un archivo **zip** conteniendo únicamente los archivos antes mencionados, sin carpetas internas ni otros archivos. De lo contrario, la entrega no será validada por la plataforma.

7. Anexos

7.1. FAQ

En este link encontrarán el documento de FAQ del TP, donde se irán cargando dudas realizadas con sus respuestas.

Todo lo que esté en ese documento es válido y oficial para la realización del TP.

7.2. Uso de la biblioteca utiles.h

La cátedra brindará a los alumnos una biblioteca que facilitará la creación de las vigas en el juego.

Deben usarla a la hora de inicializar el juego y sirve para crear las posiciones de todas las vigas que habrá en el terreno.

```
#ifndef __UTILES__
#define __UTILES__

#define MAX_VIGAS 500
#include "simpsoncraft.h"


/*
 *  Funcion que recibe un vector de vigas y su tope.
 *  Inicializa el vector con las vigas de algún mapa.
*/
void obtener_mapa(coordenada_t vigas[MAX_VIGAS], int* tope);

#endif
```

7.3. Obtención de números aleatorios

Para obtener números aleatorios debe utilizarse la función **rand()**, la cual está disponible en la biblioteca **stdlib.h**.

Esta función devuelve números pseudo-aleatorios, esto quiere decir que, cuando uno ejecuta nuevamente el programa, los números, aunque aleatorios, son los mismos.

Para resolver este problema debe inicializarse una semilla, cuya función es determinar desde dónde empezarán a calcularse los números aleatorios.

Los números arrojados por rand() son enteros sin signo, generalmente queremos que estén acotados a un rango (queremos números aleatorios entre tal y tal).

Para esto, podemos obtener el resto de la división de rand() por el valor máximo del rango que necesitamos.

Aquí dejamos un breve ejemplo de como obtener números aleatorios entre 10 y 30.

```
#include <stdio.h>
#include <stdlib.h> // Para usar rand
#include <time.h> // Para obtener una semilla desde el reloj

int main() {
    srand ((unsigned)time(NULL));
    int numero = rand() % 20 + 10; // amplitud del rango es 20 y valor mínimo
es 10.
    printf("El valor aleatorio es: %i\n", numero);
    return 0;
}
```

7.4. Limpiar la pantalla durante la ejecución de un programa

Muchas veces nos gustaría que nuestro programa pueda verse siempre en la pantalla sin ver texto anterior.

Para esto, podemos utilizar la llamada al sistema clear, de esta manera, limpiaremos todo lo que hay en nuestra terminal hasta el momento y podremos dibujar la información actualizada.

Y se utiliza de la siguiente manera:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Escribimos algo\n");
    printf("que debería\n");
    printf("desaparecer...\n");

    system("clear"); // Limpiamos la pantalla

    printf("Solo deberiamos ver esto...\n");
    return 0;
}
```

7.5. Distancia manhattan

Para obtener la distancia entre 2 puntos mediante este método, se debe conocer a priori las coordenadas de dichos puntos.

Luego, la distancia entre ellos es la suma de los valores absolutos de las diferencias de las coordenadas. Se ve claramente en los siguientes ejemplos:

- La distancia entre los puntos (0,0) y (1,1) es 2 ya que: |0 - 1| + |0 - 1| = 1 + 1 = 2
- La distancia entre los puntos (10,5) y (2,12) es 15 ya que: |10 - 2| + |5 - 12| = 8 + 7 = 15
- La distancia entre los puntos (7,8) y (9,8) es 2 ya que: |7 - 9| + |8 - 8| = 2 + 0 = 2

Recomendación: Para calcular los valores absolutos en C se puede utilizar la función `abs()`, que se encuentra en la biblioteca `stdlib.h`.

La función `abs()` toma un número entero como parámetro y devuelve su valor absoluto, es decir, convierte cualquier número negativo en positivo y deja los positivos igual. Por ejemplo:

- `abs(5)` devuelve `5`
- `abs(-3)` devuelve `3`

7.6. Emojis y colores (OPCIONAL)

A continuación les dejamos un video que explica cómo agregar emojis y un ejemplo de cómo poner colores en su trabajo práctico para que se vea más atractivo en la terminal. Cabe aclarar que esto NO reemplaza las convenciones que les damos, estas decoraciones sólo se van a usar a la hora de imprimir el mapa.

- Emojis y ASCII Art
- Colores

Además, no es necesario incluir emojis o colores al trabajo para aprobar, es 100% opcional.